HOTELPRO

Full Stack Hotel Booking and Management Web Application

PROJECT DOCUMENTATION

Technology Stack: React.js • Vite • Node.js • Express.js • MongoDB Atlas • Mongoose

Prepared for academic project submission

Project Overview

HotelPro is a full-stack hotel booking and management web application developed using the MERN stack. It aims to make hotel room browsing and booking easier while providing interfaces for viewing bookings and managing room information. The frontend uses React.js and Vite; the backend uses Node.js and Express.js; and MongoDB Atlas stores application data through Mongoose.

Project Scenario

A customer opens the HotelPro website, explores room information, selects a room, and enters a name, check-in date, check-out date, and guest count. The frontend sends the booking request to the backend, which processes it and stores the data in MongoDB when the request succeeds. The interface is intended to show a confirmation message and provide a View Bookings section. A separate room-management interface supports adding or managing room information.

System Requirements — Software

- Operating system: Windows 10/11.
- Node.js runtime and npm package manager.
- React.js and Vite for the frontend.
- Node.js and Express.js for the backend and REST APIs.
- MongoDB Atlas as the cloud database and Mongoose for database connectivity and data modeling.
- Visual Studio Code for development; Google Chrome for browser testing.
- Git for source-code version control and GitHub for hosting source code and documentation.

System Requirements — Hardware

- Processor: Intel Core i5 or equivalent recommended.
- RAM: 8 GB minimum recommended for development tools.
- Storage: Sufficient free disk space for project files and installed dependencies.
- Internet connection for MongoDB Atlas and GitHub access.
- A standard laptop or desktop display suitable for software development.

Epic 1 — Project Architecture

HotelPro follows a client-server architecture with three main layers: frontend, backend, and database.

Technical Architecture

Layer / Component	Technology	Responsibility
User / Browser	Web browser	Displays the HotelPro interface and accepts user input.
Frontend	React.js + Vite	Displays room information, booking forms, and booking records.
Backend	Node.js + Express.js	Receives requests, runs application logic, and exposes REST API routes.
Data access	Mongoose	Connects the backend to MongoDB and provides schema-based data modeling.
Database	MongoDB Atlas	Stores application data persistently.

Architecture flow: User / Web Browser → React.js + Vite → Node.js + Express.js → Mongoose → MongoDB Atlas.

ER Diagram — Entities and Relationship

The data design includes Room and Booking entities. Confirm field names, types, and relationship cardinality from the final Mongoose model files before treating this as the definitive schema.

Entity	Possible attributes
Room	Room ID; name; description; price; image; capacity; room type (if present in the schema).
Booking	Booking ID; guest name; room reference or room information; check-in date; check-out date; number of guests; total price; booking status.

Relationship: a booking is associated with a room. Verify whether the schema stores a room reference and whether one room can have many booking records before assigning exact cardinality in the ER diagram.

User Flow

1. The user opens the HotelPro website.
2. The application displays available room information.
3. The user explores room details and selects a room.
4. The user opens the booking form and enters the required details.
5. The user clicks Confirm Booking.
6. The frontend sends a request to the backend.
7. The backend processes the request and saves the booking when successful.
8. The interface displays a booking success message when the request succeeds.
9. The user opens View Bookings to access booking records.
10. The user can separately open Manage Rooms.

Features and Roles

- Room information display, including name, description, price, image, and capacity where available.
- Booking form and Confirm Booking action.
- Booking success message after a successful request.
- Manage Rooms and View Bookings navigation options.
- Backend API integration and MongoDB Atlas connectivity.
- Guest / Customer: browse rooms, submit booking details, and view booking information.
- Hotel Manager / Administrator: access the room-management interface and review bookings.

Implementation note: confirm each feature by testing it. The presence of a navigation item or form does not by itself prove that its operations work. Authentication and role-based authorization have not been confirmed.

MVC Pattern

The Model layer represents data and database schemas; the Controller layer handles application logic; and the Routing layer maps API endpoints to request handlers. Use a complete MVC diagram only if the final backend separates models, controllers, and routes. If logic is inside route files or server.js, describe it as a route-based Express backend instead.

Epic 2 — Project Setup and Configuration

Project Folder Structure

```
HotelPro/
■■■ Client/
■■■ Server/
■■■ .gitignore
■■■ README.md
```

Client contains the frontend application; Server contains backend code. The .gitignore file excludes unnecessary and sensitive files such as node_modules and .env.

Client Setup

The frontend uses React.js and Vite. Typical setup commands:

```
cd Client
npm install
npm run dev
```

During development, the frontend ran at http://localhost:5173.

Server Setup

The backend uses Node.js and Express.js. Typical setup commands:

```
cd Server
npm install
```

Start the backend using the script configured in Server/package.json. During development, the backend ran on port 8001. The exact start command depends on the project's scripts and entry file.

Database Configuration

1. Create a MongoDB Atlas account and deployment.
2. Configure database network access.
3. Obtain the MongoDB connection string.
4. Configure the backend to read the connection string from a local environment file or secure configuration mechanism.
5. Test the database connection.

Keep database connection strings and credentials out of public documentation and GitHub. Store them in a local .env file that is excluded by .gitignore.

Epic 3 — Backend Development

The backend provides the API layer through which the frontend communicates with the database.

Backend Responsibilities

- Start the Express server and accept frontend requests.
- Handle room-related and booking-related API requests.
- Connect to MongoDB Atlas through Mongoose.
- Perform database operations and return responses to the frontend.

Room API

The backend contains a room API route mounted under /api/room. It is used to work with room information. During development, the interface displayed sample room details such as a Deluxe Room, description, price, image, and capacity. Confirm exact HTTP methods and supported operations in the final roomRoutes file.

Booking API

The frontend requests booking information from GET /api/bookings for the Manage Bookings section. The interface is intended to show guest name, room information, check-in and check-out dates, guest count, total price, and booking status.

Booking Process

1. The user fills out the booking form.
2. The frontend sends the booking details to the backend.
3. The backend validates or processes the request according to implemented logic.
4. The booking is saved to MongoDB if the request succeeds.
5. The backend returns a response.
6. The frontend displays the result to the user.

Testing still required: create a booking, refresh the page, and verify that the same booking is returned by GET /api/bookings and appears in View Bookings.

Epic 4 — Database Development

MongoDB Atlas is used as the database. MongoDB stores records as documents in collections, while Mongoose provides schema-based data modeling and database operations.

Room Data

Field	Description
Name	Name of the room.
Description	Details about the room.
Price	Room price.
Image	Image reference or path.
Capacity	Maximum guest capacity.
Room type	Room category, if included in the schema.

Booking Data

Field	Description
Guest name	Name of the person making the booking.
Room	Associated room or room information.
Check-in / Check-out	Start and end dates of the stay.
Guests	Number of guests.
Total price	Total booking cost.
Status	Current booking status.

Database Connection and Validation

The backend uses Mongoose to connect to MongoDB Atlas. After configuring network access, the backend reported a successful connection during development. For final validation, create a booking and confirm that a record is present in the relevant MongoDB collection. Verify field names, types, validation rules, and relationship details from the actual model files.

Epic 5 — Frontend Development

Home Page

The home page introduces HotelPro and provides access to room information. Room cards may display room names, descriptions, prices, images, and capacity.

Booking Form

The booking form collects required stay details. The user submits them using Confirm Booking. A success message should appear only when the booking request succeeds.

Manage Rooms

The Manage Rooms navigation option opens the room-management section, which includes an interface titled Add a New Room. Confirm which create, edit, and delete operations are functional in the final version.

View Bookings

The View Bookings navigation option opens the Manage Bookings section, designed to display saved records. Test the list against the backend API and verify that actual saved bookings appear.

Frontend and Backend Communication

The React frontend communicates with the Express backend using HTTP requests. The frontend's request to /api/bookings is intended to retrieve booking data from the server rather than relying solely on hard-coded values.

Epic 6 — Project Execution

Step 1: Start the Frontend

Open the HotelPro folder in Visual Studio Code. In a terminal, run:

```
cd Client
npm install
npm run dev
```

Open the local URL printed in the terminal. During development, this was http://localhost:5173.

Step 2: Start the Backend

Open a second terminal and run:

```
cd Server
npm install
```

Start the server using the script configured in Server/package.json. The backend ran on port 8001 during development.

Step 3: Configure Environment Variables

Create a local Server/.env file and use the exact variable names expected by the backend source code. Do not publish the actual database URI, passwords, or secret keys.

Step 4: Test the Application

1. Open the HotelPro website.

2. Confirm that room information is displayed.
3. Open the booking form and enter valid booking details.
4. Click Confirm Booking and check the response.
5. Open View Bookings and verify that the saved booking appears.
6. Check the backend terminal for errors.
7. Verify the relevant record in MongoDB Atlas.

Step 5: Source-Code Hosting

The project is hosted in a GitHub repository named HotelPro. The repository should include frontend and backend source code, README files, and phase-wise documentation. It should exclude node_modules, .env files, and sensitive database credentials.

Output Screenshots with Explanation

Add genuine screenshots from the running application under the following headings. No application screenshots were supplied with this document, so the sections below are placeholders for the final submission.

Screenshot 1: HotelPro Home Page

Explanation: Shows the main interface and room information.

Screenshot 2: Room Information

Explanation: Demonstrates room name, description, price, image, and capacity.

Screenshot 3: Booking Form

Explanation: Shows the form used to enter booking details.

Screenshot 4: Booking Confirmation

Explanation: Shows the message displayed after a booking request succeeds.

Screenshot 5: View Bookings

Explanation: Shows Manage Bookings and a real saved booking after verification.

Screenshot 6: Manage Rooms

Explanation: Shows the room-management section.

Screenshot 7: MongoDB Atlas

Explanation: Demonstrates the database connection and stored records; hide credentials and sensitive data.

Screenshot 8: Application Running in VS Code

Explanation: Shows the development environment and running frontend/backend terminals.

Limitations

- Authentication and secure administrator access need to be verified or implemented.
- Validate booking dates and guest details.
- Check room availability before confirming a booking.
- Prevent overlapping bookings for the same room.
- Complete and test room creation and management operations.
- Verify that saved bookings appear correctly in the bookings list.
- Consider booking cancellation and status updates.
- Integrate a real payment gateway only if required; real online payments have not been confirmed as implemented.
- Deploy the frontend and backend if a live application is required.

Future Enhancements

1. User registration and login.
2. Administrator authentication and role-based access.
3. Room availability search by dates and guest count.
4. Booking cancellation and modification.
5. Improved form validation and error handling.
6. A complete room-management system.
7. A secure online payment gateway.
8. Email confirmation after successful booking.
9. A responsive mobile-friendly interface.
10. Deployment of the frontend and backend for public access.

Conclusion

HotelPro is a full-stack hotel booking and management web application developed using React.js, Vite, Node.js, Express.js, MongoDB Atlas, and Mongoose. It demonstrates the integration of frontend components, backend APIs, and database connectivity. During development, the frontend was run locally, the backend connected to MongoDB Atlas, room information was retrieved, and the booking flow reached a successful submission state.

The project provides practical experience in web application development, API communication, database operations, and source-code version control. Before final submission, test the complete application end-to-end, add genuine screenshots, verify the public repository, and include a live demo link if the application is deployed.

Project Links

- GitHub Repository: https://github.com/tanwarnitisha167-beep/HotelPro
- Live Demo: Add the deployment URL after the application has been deployed.